PYTHON WEB SCRAPING INTERVIEW ASSIGNMENT

Multi-Source Web Scraping & Data Consolidation

Medium-Level Take-Home Technical Assessment

1. Assignment Overview

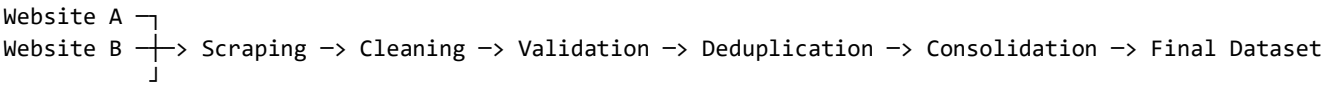
Build a Python-based web scraping and data-processing pipeline that collects information from multiple public web sources, cleans and standardizes the collected data, identifies duplicates, validates the results, and produces one consolidated dataset.

This assignment is designed to evaluate practical Python, web scraping, data processing, problem-solving, error handling, and code-quality skills at a medium level.

2. Assignment Scenario

You are working as a Python Data Scraper responsible for collecting information from multiple websites. Each website presents information in a different structure. Your job is to build a reusable process that can collect the information, transform it into a common structure, validate it, and generate a final dataset.

The expected workflow is:



3. Data Sources

Use the following public scraping practice websites:

Source 1 – Books to Scrape: https://books.toscrape.com/

The site contains book records with fields such as title, price, availability, rating, category and product URL, distributed across multiple pages.

Source 2 – Quotes to Scrape: https://quotes.toscrape.com/

The site contains quotes, authors, tags and author-related information across multiple pages.

These are public scraping practice sites. Do not attempt to bypass authentication, CAPTCHAs, robots restrictions, access controls, or other security mechanisms.

4. Objective

The completed solution should be able to:

- Collect data from both sources.
- Handle pagination.
- Normalize different source structures into a common data model.
- Clean and validate the collected data.
- Detect and remove or flag duplicates.
- Preserve source and source URL information.
- Handle failures without unnecessarily stopping the full process.
- Generate a final consolidated dataset and summary.
- Provide clear documentation and reproducible setup instructions.

5. Technical Requirements

5.1 Scraping

- Use Python.
- Use an appropriate scraping library such as Requests + BeautifulSoup, Scrapy, Playwright, or Selenium where justified.
- Scrape multiple pages from each source.
- Implement pagination rather than hard-coding a small number of pages.
- Extract the required information from each source.
- Capture the source name and original source URL.
- Handle missing HTML elements without crashing the complete scraper.

5.2 Suggested Fields

Create a standardized schema suitable for both sources. For example:

```
source
source_url
name_or_title
category
price
rating
author
tags
description
scraped_at
```

Not every field will exist for every source. Use appropriate null/empty values when a field does not apply. Do not invent data.

5.3 Data Cleaning & Standardization

- Remove unnecessary whitespace.
- Normalize text where appropriate.
- Convert prices to numeric values.
- Standardize rating values.

- Normalize missing values.
- Normalize or validate URLs.
- Remove clearly invalid records.
- Keep the transformation logic separate from the scraping logic where practical.

5.4 Duplicate Detection

Implement a reasonable duplicate-detection strategy. Exact string comparison alone may not be sufficient because the same value can appear with differences in whitespace, capitalization, or formatting.

```
"Example Book Title"
" Example Book Title "
"EXAMPLE BOOK TITLE"
```

Explain in the README which fields or normalization rules you used to determine whether records are duplicates. If you choose to flag duplicates rather than delete them, explain why.

5.5 Validation

Add basic validation before writing the final dataset. Examples:

- Required fields are present where applicable.
- Price values are numeric when a price exists.
- Ratings are within the expected range.
- URLs are present and valid-looking.
- Records have a recognizable source.
- Duplicate counts are measurable.

5.6 Error Handling & Logging

Handle common scraping failures such as:

- Connection failures
- HTTP errors
- Timeouts
- Missing page elements
- Unexpected data formats
- Invalid values
- Individual page failures

The scraper should continue processing other pages or sources when reasonably possible. Add logging so that a reviewer can understand what happened during execution.

6. Required Step-by-Step Implementation

Step 1 – Explore the Sources

Inspect both websites and understand their page structure, pagination mechanism, available fields, and differences between the sources. Document important observations.

Step 2 – Design the Data Model

Define a common output structure that can represent data from both sources. Decide how missing or source-specific fields will be handled.

Step 3 – Build Source Scrapers

Create separate scraper modules/classes/functions for Books to Scrape and Quotes to Scrape. Keep source-specific selectors and parsing logic isolated.

Step 4 – Implement Pagination

Automatically move through the available pages. Avoid manually listing every page URL.

Step 5 – Create Cleaning Functions

Create reusable functions for whitespace cleanup, numeric conversion, text normalization, URL handling, and missing-value handling.

Step 6 – Add Validation

Validate records before they enter the final dataset and record validation failures where useful.

Step 7 – Implement Deduplication

Create a repeatable method for identifying duplicate or equivalent records and document the logic.

Step 8 – Consolidate Data

Combine the cleaned records from both sources into one standardized dataset while preserving source information.

Step 9 – Add Logging and Error Handling

Add meaningful logs, exception handling, and reasonable retry behavior where appropriate.

Step 10 – Generate Output

Produce the final CSV and a summary report showing what was scraped, cleaned, rejected, and deduplicated.

Step 11 – Test the Solution

Run the complete pipeline from a clean environment and verify the output. Include tests if implemented.

7. Expected Output

The submission should produce at minimum:

```
output/
    final_dataset.csv
    summary_report.json
```

The final dataset should contain standardized records and, at minimum, enough information to identify the source and original URL.

The summary should include metrics such as:

- Total records collected per source
- Total records after cleaning
- Records rejected during validation
- Duplicate records detected/removed/flagged
- Final record count
- Execution time if available

8. Example Final Dataset

```
source | name_or_title | category | price | rating | author | tags | source_url | scraped_at
Books to Scrape | ... | ... | ... | ... | ... | ... | ... | ...
Quotes to Scrape | ... | ... | ... | ... | ... | ... | ... | ...
```

9. Recommended Project Structure

```
scraping_assignment/
├── scrapers/
│   ├── books_scraper.py
│   └── quotes_scraper.py
├── processing/
│   ├── cleaning.py
│   ├── validation.py
│   └── deduplication.py
├── output/
├── logs/
├── tests/
├── main.py
├── requirements.txt
├── README.md
└── AI_USAGE.md
```

The structure above is only a recommendation. A different structure is acceptable if it is clean, logical, and explained.

10. AI Usage – Allowed and Expected

AI tools are explicitly allowed for this assignment. Candidates may use ChatGPT, Claude, GitHub Copilot, Cursor, Gemini, or another AI coding assistant.

AI may be used for:

- Understanding HTML/page structure
- Generating initial code
- Debugging errors
- Improving scraper logic
- Designing data models
- Generating tests
- Improving documentation
- Finding edge cases

- Refactoring code

The candidate remains responsible for the final implementation. AI-generated code must be reviewed, tested, and corrected where necessary. The candidate should not submit code they cannot explain.

11. AI_USAGE.md – Required

Include an AI_USAGE.md file with the following:

- AI tools used.
- What each tool was used for.
- A few representative prompts.
- Which parts of the code were AI-assisted.
- Important changes made after reviewing AI output.
- Any incorrect or incomplete AI-generated suggestions discovered.
- How the final solution was tested and verified.

```
Example:
Tool: ChatGPT
Used for: Initial pagination approach and debugging a selector issue.
Verification: Tested pagination against all available pages and manually checked sample records.
```

12. README.md – Required Content

- Assignment/project overview
- Python version
- Installation/setup instructions
- Dependencies
- How to run the scraper
- How pagination works
- Data model
- Cleaning approach
- Validation approach
- Deduplication approach
- Error-handling approach
- Output description
- Assumptions
- Known limitations
- AI usage summary

13. Deliverables

Submit one ZIP file containing:

- Complete Python source code
- requirements.txt
- README.md

- AI_USAGE.md
- Final consolidated dataset
- Summary report
- Sample execution logs
- Tests, if implemented
- Configuration files, if used

14. Bonus Features

The following are optional and can provide additional credit:

- MongoDB or PostgreSQL integration
- Retry mechanism with exponential backoff
- Configurable settings
- Unit tests
- Parallel or asynchronous processing
- Rate limiting
- Incremental scraping
- Checkpoint/resume functionality
- Docker setup
- Data-quality report
- CLI arguments for controlling the scraper

15. Evaluation Criteria

Evaluation Area	Weight
Python & Code Quality	20%
Web Scraping	20%
Data Cleaning & Standardization	15%
Duplicate Detection	15%
Error Handling & Reliability	10%
Documentation & Project Structure	10%
AI Usage & Understanding	10%

16. Interview Follow-Up

After submission, the candidate may be asked to walk through the project and explain the implementation.

- Why did you choose your scraping library?
- How does pagination work?
- How did you handle missing fields?
- How did you handle failed requests?
- How does your duplicate-detection logic work?
- Why did you choose your standardized schema?
- What assumptions did you make?
- Which parts were AI-assisted?

- Did AI-generated code create any problems?
- How did you verify that the final result was correct?
- What would you change if this scraper had to run regularly in production?

17. Important Rules

- Use only publicly accessible information from the provided practice websites.
- Do not bypass authentication, CAPTCHA, access controls, or security mechanisms.
- Respect reasonable request rates and avoid unnecessarily aggressive traffic.
- Do not include secrets, API keys, passwords, or personal credentials in the submission.
- AI usage is allowed, but the candidate must understand and be able to explain the submitted work.
- The final submission should be reproducible from the README instructions.

18. Expected Level

This is a medium-level assignment. You are not expected to build a large production scraping platform. The focus is on writing clean Python, correctly scraping multiple sources, handling different data structures, cleaning and consolidating data, dealing with failures, and demonstrating practical problem-solving.